Ingeniería y Calidad de Software

Resumen teórico para el primer parcial

Unidades 1, 2 y 3 · Incluye «No Silver Bullet» al final de la Unidad 1

Elaborado a partir de las ocho presentaciones de cátedra indicadas y del paper de Frederick P. Brooks. Incluye ejemplos, distinciones para el examen y fuentes primarias de apoyo.

Mapa de estudio
Unidad 1: fundamentos, procesos, proyectos, ciclos de vida y Brooks.
Unidad 2: producto, requerimientos ágiles, historias, estimación y Scrum.
Unidad 3: gestión de configuración de software.

Septiembre de 2026

Unidad 1. Fundamentos de Ingeniería de Software

1. Qué es la Ingeniería de Software

Software es más que código ejecutable: incluye programas, datos, reglas, procedimientos, documentación y otros artefactos necesarios para operar y mantener un sistema. Puede ser software de sistema, utilitario o de aplicación. Una forma útil de entenderlo es como **conocimiento representado en distintos niveles de abstracción**: necesidades del negocio, requerimientos, modelos, diseño, código y pruebas [P01].

Ingeniería de Software (IS) es la aplicación de un enfoque sistemático, disciplinado y cuantificable al desarrollo, operación y mantenimiento del software. Abarca desde concebir el problema hasta operar y evolucionar el producto. Su objetivo no es solamente que el programa funcione hoy: busca que satisfaga necesidades reales, tenga calidad, pueda cambiar y se entregue con un uso responsable de tiempo y recursos [P01].

El software se diferencia de la manufactura: cada solución tiene mucho trabajo intelectual propio; copiarlo cuesta poco, pero diseñarlo cuesta; no se desgasta físicamente, aunque se deteriora por cambios del entorno y modificaciones mal integradas; no está gobernado directamente por las leyes físicas y sus fallas pueden provenir de conceptos equivocados, no solo de errores de codificación [P01].

2. Antecedentes, estado actual y crisis del software

La expresión **ingeniería de software** se difundió en las conferencias de la OTAN de 1968-1969 ante proyectos que crecían en tamaño y complejidad más rápido que la capacidad de gestionarlos. Surgieron enfoques estructurados, herramientas, estándares de proceso, gestión de calidad y, más tarde, prácticas ágiles. La «crisis del software» describe dificultades persistentes: retrasos, sobrecostos, fallas, documentación insuficiente, sistemas rígidos y productos que no satisfacen al usuario [P01].

La crisis no significa que todo proyecto fracase. Señala que **programar bien no alcanza**: también hay que comprender necesidades, coordinar personas, estimar, gestionar cambios, probar, desplegar y aprender. Las diapositivas resaltan como factores favorables el involucramiento del usuario, el apoyo de la dirección, requerimientos claros, planificación realista, hitos y personas competentes; entre los factores adversos, requerimientos incompletos o cambiantes sin control, expectativas irreales y escasez de recursos [P01].

Ejemplos de proyectos grandes y sus lecciones. La misión **Ariane 5, vuelo 501** (1996) se perdió por errores de especificación y diseño en el software de referencia inercial; las revisiones y pruebas no cubrieron suficientemente la interacción del sistema completo [E1]. La **Mars Climate Orbiter** (1999) se perdió por una incompatibilidad de unidades inglesas y métricas en software de navegación terrestre; también fallaron la validación de interfaces y la comunicación entre equipos [E2]. Como casos exitosos, el computador de guiado de **Apollo 11** cumplió su función durante la misión, dentro de un programa mucho más amplio de ingeniería, integración y operaciones [E3]. La **Worldwide LHC Computing Grid** del CERN distribuye y procesa datos de los experimentos del LHC mediante una infraestructura mundial en operación sostenida [E4]. La enseñanza no es atribuir éxito o fracaso a una sola línea de código: importan arquitectura, interfaces, pruebas, coordinación y objetivos.

3. Disciplinas que conforman la IS

- **Construcción:** requerimientos; análisis y diseño; implementación; pruebas; despliegue y mantenimiento. Cada etapa transforma conocimiento y produce artefactos verificables.
- **Gestión:** planificación, estimación, seguimiento, control de cambios, riesgos, recursos y métricas. Ordena decisiones bajo restricciones de alcance, tiempo y costo.
- **Soporte y calidad:** gestión de configuración (SCM), aseguramiento de calidad, verificación, validación y mejora del proceso. Son actividades transversales, no una etapa final [P01, P02].

Verificación pregunta «¿construimos el producto de acuerdo con la especificación?»; **validación** pregunta «¿construimos el producto que resuelve la necesidad?». Un sistema puede pasar todas las pruebas contra una especificación y aun así ser inútil si esa especificación era equivocada.

4. Proceso, proyecto, producto y ciclos de vida

Un **proceso de software** es el conjunto organizado de actividades, métodos, prácticas, personas y herramientas para desarrollar o mantener software. Es una pauta reutilizable de «cómo trabajar». Para poder administrarlo, debe hacerse explícito. Un **proyecto** es una iniciativa temporal, única, con objetivos, recursos y tareas interrelacionadas; instancia y adapta ese proceso a un contexto. El **producto** es el resultado que se usa, opera y evoluciona. Relación central: **proceso adaptado + personas y herramientas → proyecto → producto**, y el aprendizaje del proyecto mejora el proceso [P01, P06].

El **ciclo de vida del proyecto** va desde el inicio hasta su cierre. El **ciclo de vida del producto** incluye idea, desarrollo, operación, mantenimiento, nuevas versiones y retiro; normalmente supera la duración de un proyecto. Un producto puede atravesar varios proyectos. Un **modelo de ciclo de vida o de proceso** representa fases y orden de trabajo (por ejemplo, requerimientos, diseño, construcción y prueba). No equivale al proceso completo: el modelo organiza el flujo; el proceso especifica actividades, responsabilidades, técnicas, entregables y controles [P01, P06].

Fases y actividades no son sinónimos. En un modelo secuencial las actividades se agrupan en fases; en uno iterativo, requerimientos, diseño, construcción y prueba pueden repetirse en cada ciclo. «Recursivo» describe que un mismo esquema de trabajo puede aplicarse a distintos niveles: producto, subsistema, componente o historia. «Incremental» describe la entrega progresiva de capacidades útiles; «iterativo», la revisión y mejora de una solución con aprendizaje. Con frecuencia se combinan.

5. Procesos definidos y empíricos

Un **proceso definido** supone que entradas y actividades suficientemente estables permiten predecir resultados parecidos al repetir el procedimiento. Se gestiona especificando el trabajo por anticipado y controlando el cumplimiento. Es adecuado cuando tecnología, requisitos y entorno son conocidos y la variación es baja. Ventaja: trazabilidad y previsibilidad. Límite: si la incertidumbre es alta, el plan inicial puede quedarse obsoleto [P01, P06].

Un **proceso empírico** reconoce que en problemas complejos los resultados no son totalmente predecibles. Se controla mediante **transparencia, inspección y adaptación** frecuentes: hacer visible el estado, observar el resultado real y ajustar producto y forma de trabajo. Es apropiado cuando hay aprendizaje, creatividad, cambios o incertidumbre. Ventaja: respuesta temprana al error. Límite: exige disciplina, feedback real y capacidad de decisión; empírico no significa improvisado [P01, P08].

Ejemplo: si se construye por décima vez una integración idéntica y bien documentada, un procedimiento definido puede funcionar. Si se diseña una app para un comportamiento de usuarios aún desconocido, conviene formular hipótesis, entregar un incremento pequeño, medir su uso y adaptar.

6. Modelos de ciclo de vida: ventajas, límites y elección

Secuencial o cascada. Requerimientos → diseño → construcción → prueba → entrega, con hitos y entregables por fase. Facilita presupuesto, contratos y control cuando requisitos y tecnología son estables. Su límite es que el usuario puede ver valor y detectar errores conceptuales demasiado tarde; el cambio posterior cuesta más. No implica ausencia absoluta de retrocesos, pero los encarece.

Modelo en V. Variante secuencial que relaciona cada nivel de especificación/diseño con su nivel de prueba: aceptación frente a necesidades, sistema frente a requisitos, integración frente a arquitectura, unidades frente a componentes. Aporta trazabilidad y planificación temprana de pruebas. Conserva el problema de la validación tardía si no se usan prototipos.

Iterativo. Se desarrolla una solución inicial y se revisa en ciclos. Permite descubrir errores, mejorar diseño y ajustar requisitos; requiere gestionar retrabajo y evitar iteraciones sin objetivo verificable. **Incremental.** Se entrega el producto por porciones funcionales completas; permite valor temprano y priorización, pero requiere arquitectura integrable, interfaces estables y coordinación de dependencias. En la práctica, muchos proyectos son **iterativos e incrementales**.

Prototipado. Un prototipo explora requisitos, interacción o viabilidad antes de comprometer toda la construcción. Reduce incertidumbre y mejora la conversación con usuarios. Su riesgo es tratar un experimento descartable como producto listo para operar, o confundir una interfaz convincente con calidad interna.

Espiral orientada a riesgos. Cada vuelta fija objetivos, analiza riesgos, construye/prototipa y evalúa con interesados. Conviene en proyectos grandes, costosos o de tecnología incierta. Ofrece control explícito del riesgo; requiere experiencia y puede resultar pesado para iniciativas pequeñas.

Enfoques ágiles como Scrum. Usan iteraciones cortas, un backlog priorizado y revisión frecuente de incrementos útiles. Responden bien a incertidumbre de necesidades y permiten reordenar trabajo. Necesitan colaboración sostenida, calidad técnica y disponibilidad de interesados. Scrum es un marco de gestión empírica; no sustituye por sí solo las prácticas de ingeniería.

Criterios de elección: estabilidad de requerimientos; costo de cambiar; riesgo técnico, legal y de seguridad; necesidad de entrega temprana; disponibilidad de usuarios; tamaño y distribución del equipo; dependencias; madurez tecnológica; exigencias contractuales y regulatorias. Cuando hay alta incertidumbre de valor, preferir ciclos cortos y validación; cuando el riesgo técnico domina, prototipos o espiral; cuando los requisitos son estables y se exige evidencia formal, un esquema secuencial/V puede ser conveniente. Se pueden combinar: por ejemplo, hitos regulatorios con desarrollo iterativo dentro de cada hito.

7. Componentes y administración de un proyecto de software

Un proyecto requiere **objetivos claros y alcanzables**, patrocinio, stakeholders, equipo con habilidades complementarias, alcance, proceso y ciclo de vida adaptados, plan, recursos, estimaciones, riesgos, controles y criterios de calidad. El líder coordina intereses y decisiones; el equipo comparte responsabilidad por los resultados. El plan responde **qué, cuándo, cómo y quién**, y se actualiza con evidencia [P06].

Alcance del producto son las características y funciones del resultado; se comprueba contra los requerimientos. **Alcance del proyecto** es el trabajo necesario para entregarlo; se comprueba contra el plan. La **triple restricción** vincula alcance, tiempo y costo; modificar una variable suele afectar las otras y la calidad. Por ejemplo, agregar autenticación biométrica sin ampliar plazo ni presupuesto puede reducir tiempo de prueba o desplazar otras funciones. La decisión debe explicitar el intercambio y el riesgo, no ocultarlo [P06].

Planificación: definir alcance; elegir proceso y ciclo de vida; estimar tamaño, esfuerzo, calendario y costo; identificar recursos críticos; analizar riesgos; asignar trabajo; establecer hitos, controles y métricas. **Seguimiento:** comparar plan y realidad, analizar variaciones, decidir acciones y comunicar. **Riesgo** es un evento incierto con posible impacto: se identifica, evalúa por probabilidad e impacto, se planifica una respuesta y se revisa durante todo el proyecto. Ejemplo: una API externa aún no probada puede requerir una prueba técnica temprana y una alternativa de integración [P06].

Métricas: de producto (defectos, desempeño, cobertura relevante), de proyecto (avance, esfuerzo, costo, cambios) y de proceso (tendencias agregadas entre proyectos). Deben responder decisiones concretas; medir sin propósito agrega trabajo y puede distorsionar conductas. Conviene observar tamaño entregado, esfuerzo, calendario y defectos junto con satisfacción y uso real; ninguna cifra aislada demuestra éxito [P06].

Estimación clásica de software. Estimar es pronosticar con incertidumbre; planificar es decidir qué se hará con ese pronóstico. Una estimación no es automáticamente un compromiso. Distinguir **tamaño** del producto, **esfuerzo** (trabajo en horas/persona), **duración** (tiempo calendario), **costo** y **recursos**. Más personas no reducen linealmente la duración: hay dependencias, comunicación y capacitación [P07].

Se puede **contar** componentes conocidos; usar **datos históricos y analogía**; aplicar **juicio experto**; emplear **Wideband Delphi** (estimaciones individuales, discusión de diferencias y nuevas rondas); o modelos algorítmicos cuando existan datos pertinentes. El método de tres puntos da una media ponderada orientativa: **(optimista + 4 × habitual + pesimista) / 6**. Hay que aclarar supuestos, descomponer a una granularidad útil e incluir actividades omitidas con frecuencia: pruebas, revisiones, documentación, integración, reuniones, licencias y soporte. Al principio la incertidumbre es mayor y la estimación debe revisarse al aprender [P07].

8. Frederick P. Brooks: «No Silver Bullet» (al final de la Unidad 1)

Tesis central. Brooks sostiene que ninguna tecnología o técnica de gestión, por sí sola, prometía una mejora de un orden de magnitud (aproximadamente 10 veces) en productividad, fiabilidad y simplicidad del software dentro de una década. No dice que no haya progreso: propone mejoras acumulativas y sostenidas. El ensayo fue presentado en 1986 y publicado posteriormente; su argumento debe leerse en ese contexto histórico [B].

Brooks distingue **esencia** y **accidente**. La esencia es el trabajo de concebir, especificar, diseñar y validar estructuras conceptuales complejas: datos, relaciones, funciones e interacciones. Lo accidental son dificultades de representar esas ideas en lenguajes y máquinas concretas: sintaxis, memoria escasa, compilaciones lentas o herramientas poco integradas. Si el trabajo accidental ya no representa al menos el 90 % del esfuerzo, eliminarlo por completo tampoco puede producir una mejora total de 10 veces. Esa es una explicación conceptual, no una medición universal.

Cuatro dificultades esenciales. (1) **Complejidad:** las partes de un sistema interactúan de forma no lineal; al crecer aparecen más estados, dependencias y dificultades de coordinación. (2) **Conformidad:** el software debe adaptarse a leyes, organizaciones, interfaces y sistemas preexistentes, muchas veces arbitrarios. (3) **Cambiabilidad:** usuarios, plataformas, reglas y contexto presionan para modificar incluso el software exitoso. (4) **Invisibilidad:** el software carece de una representación espacial única; distintos diagramas muestran solo aspectos parciales de su estructura [B].

Qué resolvió problemas accidentales. Los lenguajes de alto nivel alejaron al programador de registros y detalles de máquina; el tiempo compartido redujo la espera de ejecución y preservó el contexto mental; los entornos integrados facilitaron herramientas y formatos comunes. Fueron avances reales, pero sus beneficios tienen un límite cuando el problema principal pasa a ser conceptual [B].

Por qué las «balas de plata» candidatas no bastan. Brooks examina lenguajes como Ada, orientación a objetos, inteligencia artificial y sistemas expertos, programación automática, programación visual, verificación formal, herramientas y estaciones de trabajo. Cada propuesta puede ayudar, pero ninguna elimina la necesidad de decidir correctamente **qué construir** ni la complejidad inherente del diseño. Incluso una prueba formal perfecta demuestra conformidad con una especificación; si la especificación no expresa la necesidad real, el producto seguirá siendo incorrecto. La discusión sobre IA del ensayo corresponde a la tecnología de su época; extrapolarla a herramientas actuales requiere una evaluación nueva [B].

Cuatro líneas prometedoras que atacan la esencia: (1) **comprar o reutilizar** en vez de construir desde cero cuando una solución existente satisface la necesidad; (2) **refinar requisitos mediante prototipos rápidos** junto con usuarios; (3) **hacer crecer el software incrementalmente**, manteniendo siempre un sistema ejecutable; (4) **identificar y desarrollar grandes diseñadores** y cuidar la integridad conceptual. Ejemplo: antes de crear un sistema de turnos completo, probar con usuarios el flujo de reserva, integrar una solución existente si encaja y ampliar el producto solo después de comprobar su utilidad [B].

Para el parcial: no confundir «accidente» con error o incidente. La distinción es entre dificultad inherente a la naturaleza del software y dificultad contingente a una tecnología o forma de trabajo. La frase «no hay bala de plata» no desautoriza herramientas ni métodos; exige comprender qué problema resuelven y medir sus límites.

Unidad 2. Producto, requerimientos y agilidad

1. Gestión de producto: construir valor, no solo funcionalidades

La gestión de producto decide **para quién**, **qué problema** y **por qué** se construye. Parte de una visión y una propuesta de valor, formula hipótesis sobre usuarios y negocio, prioriza oportunidades, define una hoja de ruta y mide resultados. **Output** es lo entregado (por ejemplo, una pantalla de pagos); **outcome** es el cambio observable que produce (por ejemplo, más compras completadas o menos tiempo de atención). Entregar una funcionalidad correcta no garantiza que alguien la use [P05, P08].

El ciclo **construir → medir → aprender** de Lean Startup busca reducir el riesgo de desarrollar algo que nadie necesita. Se explicita una hipótesis, se elige el experimento más pequeño que pueda refutarla, se observa conducta real y se decide perseverar, ajustar o **pivotar** (cambiar una hipótesis importante de producto o mercado). Una hipótesis de **valor** pregunta si el producto resuelve un problema; una de **crecimiento**, cómo nuevos usuarios lo descubren y adoptan. Elegir métricas de uso, retención, conversión o tiempo ahorrado según la hipótesis, evitando métricas de vanidad como visitas sin contexto [P05].

MVP (producto mínimo viable) es la versión o experimento mínimo que permite aprender algo relevante sobre la propuesta de valor con usuarios reales. «Mínimo» significa sin funciones superfluas, no incompleto o inseguro. Puede ser una prueba de humo (por ejemplo, mostrar una oferta y medir interés antes de construir el servicio), un prototipo funcional o una versión acotada. **Prototipo** explora diseño o factibilidad; un MVP debe producir evidencia sobre una hipótesis de negocio. El video previo de Dropbox, citado en las diapositivas, ilustra cómo evaluar interés sin construir primero toda la infraestructura [P05].

El material diferencia otros «mínimos»: **MMF** es una funcionalidad mínima comercializable que aporta valor por sí misma; **MVF** es una funcionalidad mínima viable usada para explorar un área incierta dentro de un producto existente; **MMP** es un producto mínimo comercializable que reúne lo suficiente para un primer lanzamiento; **MMR** es una liberación mínima comercializable posterior; **MLP** (producto mínimo adorable) busca además una experiencia memorable, menor fricción y conexión emocional. Estos términos son herramientas de estrategia, no artefactos obligatorios de Scrum. Una **historia vertical** puede ser pequeña en esfuerzo técnico y aun así aportar una experiencia completa al usuario [P05].

Ejemplo integrado: una app de transporte quiere probar si los pasajeros valoran compartir el costo. MVP: ofrecer a un grupo limitado la opción de solicitar un viaje compartido y medir selección y repetición. MMF: reservar y pagar un viaje compartido completo. MMP: primer producto comercializable con funciones esenciales, soporte y fiabilidad. MLP: agregar detalles de confianza, claridad y cuidado que mejoran la experiencia. Si los usuarios expresan interés pero no repiten, se revisa la hipótesis antes de sumar funciones.

2. Filosofía ágil y requerimientos ágiles

El Manifiesto Ágil valora más **personas e interacciones** que procesos y herramientas; **software funcionando** que documentación exhaustiva; **colaboración con el cliente** que negociación contractual; **respuesta al cambio** que seguir un plan rígido. Los elementos de la derecha conservan valor. Sus principios incluyen entregas frecuentes, cooperación diaria entre negocio y desarrollo, excelencia técnica, simplicidad, equipos autoorganizados y reflexión periódica [P03, E5]. Ser ágil no es trabajar sin disciplina: es sostener ciclos de aprendizaje y calidad.

Un **requerimiento de negocio** expresa un objetivo (reducir 20 % el tiempo de atención); uno **de usuario**, una capacidad deseada (consultar el estado de un trámite); uno **funcional**, comportamiento del sistema (generar un comprobante); uno **no funcional**, una cualidad o restricción medible (respuesta menor a 2 s en condiciones definidas); uno **de implementación**, una decisión técnica impuesta (alojar en una nube determinada). Los objetivos pertenecen al dominio del problema; la solución técnica debe justificarse respecto de ellos [P03].

En gestión ágil de requerimientos se detalla **lo suficiente, cuando hace falta**. Un Product Backlog ordenado contiene trabajo de distinta granularidad: lo cercano a implementar necesita más claridad; lo lejano puede permanecer amplio. Se conversa con interesados, se reprioriza con nueva evidencia y se elimina trabajo sin valor. La comunicación cara a cara, o un canal rico con feedback rápido, ayuda a resolver ambigüedades. La documentación necesaria se conserva, pero se evita especificar de forma exhaustiva algo que probablemente cambie [P03].

3. Historias de usuario (User Stories)

Una historia es un **recordatorio de conversación** sobre una necesidad valiosa, no una especificación completa ni una tarea técnica. Sus **3 C** son **tarjeta** (descripción breve), **conversación** (descubrimiento de detalles y alternativas) y **confirmación** (criterios y pruebas que muestran cuándo se satisfizo). Formato útil: **«Como [rol], quiero [capacidad] para [beneficio]»**. El beneficio obliga a discutir el valor; la plantilla no debe forzarse cuando otro formato comunica mejor [P03].

Ejemplo: «Como conductor, quiero buscar un destino por calle y altura para llegar sin perderme». Criterios de aceptación: se informa si la dirección no existe; se muestra una ruta cuando la dirección es válida; la búsqueda responde en no más de 30 s bajo condiciones especificadas. Las pruebas derivadas cubren dirección válida, calle inexistente, altura inválida y demora excesiva. Los criterios describen límites y comportamiento esperado; las pruebas concretan entradas, pasos y resultados [P03].

Una buena historia se evalúa con **INVEST**: independiente en lo posible; negociable; valiosa; estimable; pequeña para un ciclo; y comprobable. Conviene cortar historias en **porciones verticales** que atraviesen interfaz, lógica y datos y produzcan una capacidad observable. Separarlas solo por capas («hacer la base de datos», «hacer el botón») puede crear tareas técnicas necesarias, pero no historias de usuario completas [P03].

Modelado de roles: identificar grupos con objetivos y comportamientos distintos; usar tarjetas de rol o personas para recordar contexto de uso; consultar usuarios reales siempre que sea posible. Un **proxy** (ventas, soporte, experto de dominio o gerente) puede aportar información cuando el usuario no está disponible, pero introduce el riesgo de suponer necesidades equivocadas. Las historias deben conservar la perspectiva del usuario, no confundirse con fechas, tecnologías o restricciones aisladas. Por ejemplo, «desarrollar en Java» es una restricción de implementación; «entregar el 30 de julio» es una restricción de proyecto, no una historia [P03].

Criterios de aceptación fijan límites específicos de una historia y ayudan a que Product Owner, desarrolladores y testers compartan entendimiento. La **prueba de aceptación** ejecuta casos para comprobar esos criterios. **Definition of Ready (DoR)** es un acuerdo opcional del equipo sobre claridad suficiente para iniciar trabajo; no es un elemento obligatorio de la Guía Scrum. **Definition of Done (DoD)** es el estándar de calidad compartido que debe cumplir un incremento para considerarse terminado. Una historia puede cumplir sus criterios particulares y todavía no estar «Done» si no pasó integración, pruebas o documentación requeridas por la DoD [P03, E6].

Una **spike** es una investigación acotada en tiempo para reducir una incertidumbre importante antes de estimar o implementar. Puede ser técnica (¿soporta la API el volumen previsto?) o funcional (¿entienden los usuarios la visualización propuesta?). Debe tener una pregunta, un límite y un resultado verificable; es una excepción para incertidumbres significativas, no un paso obligatorio antes de cada historia [P03].

4. Estimaciones ágiles

El tamaño relativo compara historias entre sí. Un **story point (SP)** es una unidad propia del equipo para valorar tamaño considerando trabajo, complejidad e incertidumbre; **no es una hora ni una medida universal**. Una historia de 8 SP no equivale necesariamente a ocho días. Mantener una historia de referencia y una escala estable (por ejemplo, 1, 2, 3, 5, 8, 13) permite comparar. Si una historia es demasiado grande o incierta, dividirla o investigarla [P04].

En **Planning Poker**, el equipo que hará el trabajo escucha la historia, pregunta al Product Owner, elige estimaciones en privado y revela simultáneamente las cartas. Quienes estimaron más alto y más bajo explican sus supuestos; el grupo conversa y repite hasta tener una estimación útil. El valor del método está en descubrir diferencias de entendimiento, no en fingir precisión [P04].

Velocidad es la suma de SP de historias terminadas en un Sprint; las parcialmente completas cuentan cero. Sirve para pronosticar dentro del mismo equipo y bajo condiciones comparables, no para comparar equipos ni premiar puntos. **Capacidad** es tiempo de trabajo realmente disponible después de licencias, soporte, reuniones y otras obligaciones. Velocidad observada y capacidad ayudan a elegir una cantidad realista de trabajo; ambas se ajustan con evidencia [P04, P08].

Ejemplo de pronóstico: si quedan 90 SP de un backlog suficientemente conocido y el equipo suele terminar entre 18 y 24 SP por Sprint, un rango inicial es de aproximadamente **4 a 5 Sprints**. No es una fecha garantizada: alcance, composición del equipo, defectos y aprendizaje pueden cambiar. Si un Sprint tiene la mitad de disponibilidad habitual, se debe considerar esa capacidad menor antes de prometer 24 SP.

5. Framework Scrum

Scrum es un marco liviano para generar valor en problemas complejos mediante empirismo. Se apoya en **transparencia, inspección y adaptación** y en cinco valores: **compromiso, foco, apertura, respeto y coraje**. El equipo es pequeño, multifuncional y autogestionado. La Guía Scrum oficial vigente que referencia el curso es la versión **2020**; las diapositivas también incorporan la **Scrum Guide Expansion Pack 2026**, que añade lenguaje y roles. Conviene distinguir ambos planos en un examen [P08, E6, E7].

Responsabilidades oficiales del Scrum Team. El **Product Owner** maximiza el valor y es responsable del Product Backlog y su orden; puede delegar tareas, pero conserva la responsabilidad. El **Scrum Master** responde por la efectividad de Scrum y ayuda a eliminar impedimentos y mejorar el marco de trabajo. Los **Developers** crean cada Sprint un Incremento utilizable que cumple la DoD, planifican el trabajo y adaptan el Sprint Backlog. Stakeholders colaboran y dan feedback, pero no constituyen una cuarta responsabilidad interna del Scrum Team oficial [E6].

Sprint: contenedor de todos los eventos, de **un mes o menos**, con un **Sprint Goal** que da dirección. Dentro del Sprint pueden aclararse y renegociarse detalles del alcance con el Product Owner, siempre preservando el objetivo y la calidad. Cada Sprint debe producir al menos un incremento utilizable; no se necesita esperar a una «fase final de endurecimiento» para integrar y probar [E6].

Eventos y tiempos máximos de la Guía 2020: Sprint Planning (hasta 8 h para un Sprint de un mes) responde por qué vale la pena el Sprint, qué puede hacerse y cómo se hará; **Daily Scrum** (15 min) permite a Developers inspeccionar avance hacia el Sprint Goal y adaptar el plan; **Sprint Review** (hasta 4 h) inspecciona resultado y contexto con stakeholders y adapta el Product Backlog; **Sprint Retrospective** (hasta 3 h) mejora calidad y efectividad del equipo. Para Sprints menores, Planning, Review y Retrospective suelen ser más breves. El **refinamiento del Product Backlog** es una actividad continua, no un evento formal ni una cuota fija obligatoria de 10 %; ese 10 % aparece como orientación en las diapositivas [P08, E6].

Artefactos y compromisos: Product Backlog (lista emergente, ordenada y transparente) ↔ **Product Goal** (objetivo a largo plazo); **Sprint Backlog** (Sprint Goal, ítems elegidos y plan) ↔ **Sprint Goal**; **Increment** (trabajo integrado y utilizable) ↔ **Definition of Done** (estándar de calidad). Si un ítem no cumple DoD, no integra el incremento y vuelve al Product Backlog para consideración posterior. Una liberación a usuarios puede ocurrir antes de Sprint Review; Review no es una puerta de aprobación [E6].

Planificación en varios niveles: estrategia/portafolio → producto → release → Sprint → día. Un **release** es una decisión de liberar valor, no un evento oficial de Scrum; puede agrupar varios Sprints, coincidir con cada Sprint o hacerse al terminar una funcionalidad. A nivel de producto se definen objetivos y oportunidades; a nivel de release se proyectan capacidades y riesgos; en Sprint Planning se selecciona trabajo para un objetivo cercano; en Daily Scrum se adapta el plan del día. La granularidad aumenta a medida que el trabajo se acerca [P08].

Métricas útiles: incremento funcionando y probado, resultados de uso, capacidad, velocidad y tendencias de trabajo pendiente (burndown) o entregado. Un burndown muestra trabajo restante, pero no prueba que el producto genere valor. **Running Tested Features (RTF)** cuenta funcionalidades ejecutables y probadas; expresa mejor progreso real que tareas «casi listas». Elegir pocas métricas ligadas a decisiones. La velocidad es una herramienta de planificación local, no un objetivo de productividad [P08].

Ampliación 2026 usada en clase: llama **Product Developers** a quienes crean el incremento; describe **Stakeholders** y **Supporters** (interesados que ayudan a remover barreras organizacionales); diferencia **Definition of Output Done** (calidad verificable de lo entregado) de **Definition of Outcome Done** (evidencia de beneficio real); y admite apoyo de IA bajo responsabilidad humana. Son extensiones del material 2026, **no elementos exigidos por la Guía Scrum oficial 2020**. La diapositiva usa «Product Developer humano o automatizado» dentro de esa ampliación; en la guía oficial la responsabilidad se denomina Developers y pertenece a personas del Scrum Team [P08, E6, E7].

Confusiones frecuentes: Daily Scrum no es reporte al Scrum Master; Review no es una demo sin conversación; Retrospective revisa la forma de trabajar, no prioriza producto; una historia terminada exige calidad integrada; y «hacer todo por pedacitos» sin valor, aprendizaje ni adaptación no basta para ser ágil.

Unidad 3. Gestión de Configuración de Software (SCM)

1. Propósito y conceptos básicos

SCM es una disciplina transversal que identifica los elementos del software, controla sus cambios, registra su estado y verifica la integridad de las configuraciones durante todo el ciclo de vida. Su propósito es saber **qué versión de qué artefacto** conforma un producto, **por qué cambió**, **quién autorizó el cambio** y **cómo reproducir una entrega**. Abarca más que hacer `commit` o `update`: también involucra decisiones, trazabilidad, auditorías y liberaciones [P02].

Un **ítem de configuración (IC)** es cualquier artefacto del producto o proyecto que deba compartirse, cambiarse y seguirse: requerimientos, arquitectura, código, casos de prueba, scripts, dependencias, configuración de despliegue, manuales, planes o registro de defectos. Una **versión** es el estado particular de un IC en un momento o contexto; una **variante** es una línea alternativa que evoluciona por separado, por ejemplo una edición para otro sistema operativo. La **configuración del software** es el conjunto específico de IC y versiones que forman un sistema en un momento dado [P02].

Un **repositorio** conserva IC, historia, atributos y relaciones, y permite evaluar el impacto de cambios. En un repositorio **centralizado** hay una copia principal en un servidor; el control es directo, pero el servidor es un punto crítico. En uno **distribuido**, cada clon contiene la historia completa, favoreciendo trabajo local y distintos flujos de integración. Ninguna topología reemplaza políticas de acceso, respaldo y revisión [P02].

Una **línea base (baseline)** es una configuración revisada y acordada que sirve como referencia; cambiarla requiere un procedimiento controlado. No es simplemente «la última versión» ni necesariamente una liberación comercial. Una **etiqueta** puede identificarla para reconstruir el estado. Una **release** es un paquete entregado a usuarios o producción, que debe poder vincularse a una línea base, artefactos construidos, pruebas y notas de versión [P02].

Una **rama (branch)** permite desarrollo paralelo o experimental. **Merge** integra cambios; si dos ramas modificaron el mismo contenido incompatible, aparece un conflicto que debe resolverse y probarse. Ramas demasiado largas aumentan divergencia y costo de integración. Una rama se integra o se descarta de forma explícita [P02].

2. Actividades fundamentales de SCM

Identificación de configuración: definir qué IC se controlan, sus nombres unívocos, estructura, propietarios, relaciones y criterios de versionado. Ejemplo: para la versión 2.1 de una app registrar requerimientos aprobados, commit de código, versión de esquema de base de datos, dependencias y pruebas [P02].

Control de cambios: una solicitud describe motivo y artefactos afectados; se analiza impacto en requisitos, interfaces, costo, plazo, pruebas y riesgos; se aprueba, rechaza o pospone; se implementa y revisa; se actualiza la línea base y se informa la decisión. En entornos formales puede intervenir un **Comité de Control de Cambios (CCB)** con representantes de áreas involucradas. La rigurosidad debe adecuarse al riesgo: una corrección menor no necesita la misma ceremonia que una modificación de seguridad o contrato [P02].

Registro e informe de estado: responder qué IC existe, qué versión está aprobada, qué cambios están pendientes, aprobados o implementados, qué release los contiene y qué pruebas los validaron. La trazabilidad conecta solicitud → requisito → cambio de código → prueba → entrega. Automatizar reportes reduce errores manuales [P02].

Auditoría de configuración: la **funcional (FCA)** verifica que comportamiento y rendimiento reales se correspondan con los requisitos; la **física (PCA)** comprueba que artefactos, versiones y documentación declarados estén realmente presentes y consistentes. Ambas complementan verificación y validación, y ayudan a reproducir y mantener el producto [P02].

El **plan de SCM** establece alcance de IC, convenciones, herramientas, responsabilidades, flujo de cambios, reglas de ramas y baselines, auditorías, informes y procedimientos de entrega. Sin esas reglas, aparecen pérdidas de cambios, superposición de trabajo, ejecutables que no corresponden al código fuente, regresiones y versiones imposibles de reconstruir [P02].

3. SCM en ambientes ágiles

La agilidad necesita SCM **ligera, visible y automatizada**: integración frecuente, pruebas automáticas, revisión de cambios, construcción reproducible y trazabilidad suficiente. El equipo comparte la responsabilidad; la gestión de configuración se integra al trabajo del Sprint y a la DoD. Responder rápido al cambio no equivale a aceptar cambios sin conocer su impacto. Un CCB formal puede ser sustituido o simplificado para cambios ordinarios, mientras cambios regulados o de alto riesgo conservan aprobación explícita [P02].

Ejemplo completo: el cliente pide corregir un cálculo de descuento. Se registra la solicitud y la versión afectada; se analiza si cambia una regla de negocio; se modifica una rama corta; se agregan pruebas de la regla; se integra tras revisión y ejecución automática; se etiqueta el incremento y se anota en la release. Si aparece una regresión, la trazabilidad permite identificar el cambio y reconstruir la versión anterior.

Repaso final: relaciones que suelen evaluarse

- **Proceso / proyecto / producto:** patrón reutilizable / iniciativa temporal / resultado que continúa evolucionando.
- **Ciclo de vida del proyecto / del producto:** el segundo suele incluir varios proyectos y operación prolongada.
- **Definido / empírico:** control por plan predictivo / control por transparencia, inspección y adaptación.
- **Iteración / incremento:** revisar y aprender / agregar capacidad útil; pueden coexistir.
- **Alcance del producto / del proyecto:** funciones del resultado / trabajo para entregarlo.
- **Tamaño / esfuerzo / duración:** magnitud del trabajo / trabajo humano / tiempo de calendario.
- **Criterio de aceptación / DoD:** condición de una historia / estándar común del incremento.
- **MVP / MMP / MLP:** aprender / lanzar comercialmente / cuidar una experiencia memorable.
- **Story points / velocidad / capacidad:** tamaño relativo / puntos terminados por Sprint / disponibilidad real.
- **Baseline / release:** referencia controlada / entrega a destinatarios.
- **Verificación / validación:** cumplimiento de especificación / solución de necesidad.

Fuentes y alcance

Material de cátedra: [P01] PRE_01_IntroducciónIngenieriaDeSoftware.pdf; [P02] PRE_02_SCM.pdf; [P03] PRE_03_RqsAgilesUSERSTORIES.pdf; [P04] PRE_04_EstimacionesÁgiles.pdf; [P05] PRE_05_GestionDeProductos.pdf; [P06] PRE_06_ComponentesDeProyectoSW.pdf; [P07] PRE_07_EstimacionesDeSW.pdf; [P08] PRE_08_SCRUMPlanificacionReleaseSprintMetricas.pdf. Se resumieron las ocho presentaciones indicadas para el parcial, incluida la presentación P05 revisada visualmente porque la mayor parte de su texto está en imágenes.

Paper: [B] Frederick P. Brooks Jr., «No Silver Bullet—Essence and Accident in Software Engineering», texto incluido en Brooks_1986_-_No_Silver_Bullet.pdf. Esta guía presenta sus ideas en español, mediante paráfrasis, no una traducción literal íntegra.

Fuentes primarias de apoyo: [E1] ESA, informe de Ariane 501: https://www.esa.int/Newsroom/Press_Releases/Ariane_501_-_Presentation_of_Inquiry_Board_report ; [E2] NASA, Mars Climate Orbiter: https://llis.nasa.gov/lesson/641 ; [E3] NASA, Apollo 11 Mission Report: https://www.nasa.gov/wp-content/uploads/static/apollo50th/pdf/A11_MissionReport.pdf ; [E4] CERN, Worldwide LHC Computing Grid: https://home.cern/science/computing/grid/ ; [E5] Manifiesto Ágil y principios: https://agilemanifesto.org/iso/es/manifesto.html y https://agilemanifesto.org/principles ; [E6] Guía Scrum oficial 2020: https://scrumguides.org/scrum-guide.html ; [E7] Scrum Guide Expansion Pack 2026: https://scrumexpansion.org/scrum-guide-expanded/ .

Criterio editorial: se priorizaron los temas explícitos del primer parcial. Los ejemplos y aclaraciones añadidos se identifican como apoyo conceptual; no sustituyen la terminología particular que pueda exigir la cátedra.